

컴퓨터비전 개론 3주차 요약본

11. CNN(Convolutional Neural Network)

11.1 Convolutional Neural Network (CNN) 개요

- CNN이란?

- 이미지 인식 및 컴퓨터 비전에 특화된 신경망 모델
- 이미지의 공간적 구조를 유지하면서 특징을 학습하는 능력을 가짐
- 딥러닝 모델에서 영상 데이터 처리의 핵심 기술로 사용됨

11.2 주요 구성 요소

- 합성곱 층(Convolutional Layer)

- 필터(커널)를 사용해 특징 추출
- 중요한 시각적 패턴 감지 (예: 가장자리, 윤곽)

- 풀링 층(Pooling Layer)

- 특징 맵 크기 축소 (Down Sampling)
- 연산량 감소 및 중요한 정보 유지 (예: Max Pooling)

- 완전 연결 층(Fully Connected Layer, FC)

- 최종 분류 작업 수행
- Flatten을 거쳐 1차원 형태로 변환 후 출력

- 활성화 함수(Activation Function)

- 비선형성을 추가하여 복잡한 패턴 학습
- 주로 ReLU(Rectified Linear Unit) 사용

- 손실 함수(Loss Function) & 옵티마이저(Optimizer)

- 예측 값과 실제 값 차이를 줄이기 위해 사용
- 분류 문제에서는 주로 Cross-Entropy Loss 활용

11.3 CNN의 연산 과정

1. 합성곱 연산(Convolution)

- 이미지의 특정 패턴(특징)을 추출하는 과정
- 필터(커널, Kernel)를 사용하여 이미지의 영역별 특징을 학습
- 합성곱 연산 후 특징 맵(Feature Map)이 생성됨

2. 스트라이드(Stride)

- 커널이 이동하는 간격을 의미
- 큰 값일수록 출력 크기가 작아지고, 작은 값일수록 세밀한 특징을 학습 가능

3. 풀링(Pooling)

- **맥스 풀링(Max Pooling)**: 가장 큰 값을 선택하여 대표값으로 사용
- **평균 풀링(Average Pooling)**: 해당 영역의 평균값을 선택

4. 패딩(Padding)

- 커널이 경계를 넘어가면서 데이터 손실을 방지하기 위해 0을 추가하는 기법
 - 유형
 - **Valid Padding**: 패딩 없이 진행 (출력 크기가 작아짐)
 - **Same Padding**: 입력 크기를 유지하도록 패딩을 추가

5. Flattening Layer

- CNN의 마지막 합성곱 계층 출력을 1차원 배열로 변환하여 Fully Connected Layer로 전달

12. 튜닝 기법

12.1 깊은 신경망의 필요성

- 얇은 모델: 모서리·색 등 **저수준 특징**(모서리, 색상 등)만 포착 → 표현력 한계
- 깊은 모델: 저수준 → 중간 → 고수준 특징을 **계층적**으로 결합(눈·코·입 → 얼굴 전체)
- **AlexNet 이후** 더 깊은 VGG-19, ResNet(152층)로 발전
 - 실습 모델 구조
 - CNN_v2 설계 원칙(6×Conv + 3×Pool)

- 핵심: 풀링으로 이미지 크기 ½ 감소 시 → 채널 수 2배 증가
 - Block1: conv1-2 + pool1 → 32채널(기본 특징)
 - Block2: conv3-4 + pool2 → 64채널(중간 특징)
 - Block3: conv5-6 + pool3 → 128채널(고차원 특징)
- 이유: 32채널 → 64채널 → 128채널 해상도 축소에 따른(정보 손실 최소화)
- 깊은 신경망은 얇은 모델보다 더 복잡·추상적 특징을 학습

12.2 최적화 함수 (Optimizer)

- 개념
 - 모델이 학습하면서 손실 함수(Loss Function)를 최소화하는 방향으로 파라미터(가중치(w: weight), 편향(b: bias))를 조정하는 과정
- 종류
 - **SGD (Stochastic Gradient Descent, 확률적 경사 하강법)**
 - 무작위로 한 개의 데이터 샘플을 사용하여 가중치 업데이트
 - 연산량 감소, 빠른 학습 가능하지만 느린 수렴 및 최적값 주변에서 진동 가능(지역 최소값, local minimum)
 - **SGD + 모멘텀(Momentum)**
 - 이전 기울기의 영향을 반영하여 관성을 가지며 이동
 - 진동을 줄이고 더 빠르게 수렴
 - 속도(velocity)를 고려하여 관성 효과를 부여
 - **Adam (Adaptive Moment Estimation)**
 - 1차 모멘트(기울기 평균)와 2차 모멘트(기울기 제곱 평균)를 활용하여 최적화 진행
 - 학습률 자동 조정 및 안정적인 초기 수렴을 보임
 - 대부분의 경우 좋은 성능을 보이며 가장 널리 사용됨

12.3 과적합(Overfitting)

- 과적합이란?
 - 모델이 학습 데이터에 과도하게 적응하여 일반화 성능이 저하되는 현상
 - 원인

- 모델의 파라미터 수가 너무 많음
- 데이터가 부족하여 noise까지 학습
- 학습 Epoch이 너무 많아 과학습 발생
- **과적합의 신호**
 - 전형적 패턴: **train loss↓ 계속, val loss가 어느 시점부터 반등↑**
 - 즉, 검증 손실이 발생하기 시작하면 과적합 의심 징후

12.4 과적합 대응 방안

- **드롭아웃(Dropout)**
 - 학습 중 일부 뉴런 무작위 비활성화 → 특정 경로 의존 줄여 일반화↑
 - **Train 모드:** 예를 들어 $p=0.5$ 면 절반 0 처리 & 생존 뉴런 스케일 보정(값 2배)
 - **Eval 모드:** 적용 안 함(일관된 예측). `net.train()/eval()` 전환 필수
 - 실제 배치
 - 각 **MaxPool** 직후 배치
 - **분류기(Fully Connected layer)** 사이 배치
 - 깊어질수록 비율 점진 증가(예: $0.2 \rightarrow 0.3 \rightarrow 0.4$)
 - 주의 사항: 학습 시에만 적용! 평가 시에는 모든 뉴런 사용
 - `net.train()` - 드롭아웃 활성화
 - `net.eval()` - 드롭아웃 비활성화
- **배치 정규화 (Batch Normalization)**
 - 미니배치 단위로 데이터의 분포를 정규화(평균:0, 분산:1) → 분포 안정화 및 학습 효율↑
 - 권장 위치: **Conv → BN → ReLU** (ReLU로 정보 소실 전에 분포 안정화)
 - 주의 사항: 같은 BN 인스턴스 재사용 금지 → 채널 수 같아도 별도 인스턴스 필수!
 - 재사용하면 파라미터 공유로 학습 실패
- **데이터 증강(Data Augmentation)**
 - 데이터 증강이 필요한 이유

- 적은 데이터셋으로 인한 과적합 문제
- 데이터 불균형 (예: 정상 90%, 비정상 10%)
- 원본 데이터를 변형하여 데이터 양 증가, 데이터 다양화로 **일반화 향상(다양한 상황에 대응 가능)**
- **제시 기법: RandomHorizontalFlip, RandomErasing**(사각 영역 제거로 구성 요소 편향 완화)

```
transform_train = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5), # 50% 좌우 반전
    transforms.ToTensor(),
    transforms.Normalize(0.5, 0.5),
    transforms.RandomErasing(p=0.5, scale=(0.02, 0.33)) # 영역 지우
    기
])
```

13. 사전 학습 모델 활용_1

13.1 사전학습 모델(Pretrained Model)

- **정의:** ImageNet 같은 대규모 데이터셋으로 미리 훈련된 신경망 모델
- 이미 수백만 장의 이미지를 학습 → 기본적인 시각적 특징(선, 면, 질감)을 인식 가능
- 새로운 문제에 빠르고 정확하게 적응 (학습 시간 단축, 성능 향상)
- 파이토치에서는 `torchvision.models` 로 쉽게 불러올 수 있음

13.2 파인튜닝(Fine-tuning)과 전이학습(Transfer Learning)

- **파인 튜닝 (Fine-Tuning)**
 - 모든 **파라미터**를 새로운 데이터로 다시 학습
 - 데이터가 **충분히 많을 때** 적합 (예: CIFAR-10의 5만 건)
 - 모델 전체를 새로운 작업에 맞게 조정
- **전이학습(Transfer Learning)**
 - 일부 파라미터는 **고정(동결)**하고 출력 레이어만 학습
 - 데이터가 **적을 때** 더 적합

- 기존 지식을 최대한 보존
- 학습 시간이 상대적으로 짧음

13.3 적응형 평균 풀링 (AdaptiveAvgPool2d)

- 입력 이미지 크기와 무관하게 **출력 크기를 일정하게 유지**
- CNN 모델이 다양한 입력 크기에서도 작동할 수 있게 함
- **일반 MaxPool2d**: 커널 크기 지정 → 출력 크기가 입력에 따라 변함
- **AdaptiveAvgPool2d**: 출력 크기 지정 → 입력에 관계없이 일정한 출력

13.4 사전학습 모델 성능 비교

- Top-1은 1순위 예측 오답률 / Top-5는 상위 5개 내 정답 미포함 확률

모델	Top-1 오류(%)	Top-5 오류(%)	특징
AlexNet	43.45	20.91	초창기 CNN
VGG-19-BN	25.76	8.15	안정적, 깊은 구조
ResNet-18	30.24	10.92	가벼움, 잔차 연결
ResNet-152	21.69	5.94	매우 깊은 모델, 높은 성능

13.5 CIFAR-10 데이터셋을 통한 비교

- 이미지 크기 조정

- CIFAR-10 원본: 32×32 픽셀
- 사전 학습 모델 학습 크기: 224×224
- **실습에서 사용**: 112×112 (학습 시간과 성능의 균형)

데이터 변환

- 학습 데이터

```
transforms.Compose([
    transforms.Resize((112, 112)), # 리사이즈 (112×112)
    transforms.RandomHorizontalFlip(p=0.5), # 좌우 반전 (p=0.5)
    transforms.ToTensor(), # tensor로 변환
    transforms.Normalize(...), # 정규화
```

```
transforms.RandomErasing(p=0.5) # 랜덤 지우기 (p=0.5)
])
```

- 검증 데이터

```
transforms.Compose([
    transforms.Resize((112, 112)),
    transforms.ToTensor(),
    transforms.Normalize(...)
])
# batch size: 50
```

- ResNet-18

```
### 단순 구조, fc 레이어 접근이 쉬움

# 1. 사전 학습 모델 불러오기
net = models.resnet18(pretrained=True)

# 2. 최종 레이어 확인
fc_in_features = net.fc.in_features # 512

# 3. 레이어 교체 (CIFAR-10의 10개 클래스에 맞게)
net.fc = nn.Linear(fc_in_features, n_output)
```

- VGG-19-BN

```
### 파라미터가 많고 깊지만, 높은 정확도 달성 가능

# 1. 모델 불러오기
net = models.vgg19_bn(pretrained=True)

# 2. 최종 레이어 확인 (classifier의 6번째 요소)
in_features = net.classifier[6].in_features # 4096

# 3. 레이어 교체
net.classifier[6] = nn.Linear(in_features, n_output)
```

13.6 최적화 함수 비교

- Adam

- 각 파라미터에 대해 학습률을 **개별적으로 적응**
- 초반 학습 속도가 빠름
- 때로는 파인 튜닝에서 일반화 성능 저하 가능

- SGD with Momentum

- 고정된 학습률 사용
- 이전 기울기의 지수 가중 평균(모멘텀) 활용
- **파인 튜닝에서 더 안정적이고 우수한 일반화 성능**
- 부드러운 수렴

항목	Adam	SGD (with Momentum)
방식	적응형 학습률 (1차·2차 모멘트 활용)	일정 학습률 + 관성(모멘텀)
장점	빠른 수렴, 초기 학습 효율적	안정적 수렴, 일반화 성능 우수
단점	Fine-tuning 시 불안정할 수 있음	학습 초반 속도 다소 느림
권장	대규모 데이터 초반 학습	사전학습 가중치 미세조정 시

13.7 ResNet-18 vs VGG-19-BN

- ResNet-18 (현대적 아키텍처)

- **잔차 연결(Residual Connection)** 구조
- 파라미터 수가 적어 **학습 속도가 빠름**
- 가벼운 모델
- 빠른 학습과 우수한 성능

- VGG-19-BN (고전적 아키텍처)

- 합성곱과 풀링을 순차적으로 깊게 쌓음
- Batch Normalization으로 안정성 향상
- 파라미터가 많아 **학습 시간이 오래 걸림** (에폭당 약 8분)
- **더 높은 정확도 달성 가능**

13.8 핵심 내용 정리

- Fine-tuning & Transfer Learning 요약

- 사전 학습 모델을 활용하면 학습 시간 단축 + 성능 향상
- **Fine-tuning**: 모든 파라미터 재학습 → “모델 전체 최적화”
- **Transfer Learning**: 일부 동결 → “지식 보존 + 효율적 학습”
- 파인 튜닝은 데이터가 충분할 때, **전이 학습**은 데이터가 적을 때
- **SGD with Momentum**이 파인 튜닝에 효과적
- **ResNet-18**: 빠른 학습, **VGG-19-BN**: 높은 정확도
- 상황에 맞는 모델 선택이 중요

- 재현성 확보 (`torch_seed`)

```
# 난수 생성 과정을 고정(동일한 시드 설정)하여 동일한 결과 보장
import torch
torch.manual_seed(42)
torch.backends.cudnn.deterministic = True
```

- 풀링 레이어 제거

- CIFAR-10 등 작은 입력 데이터에 맞게 **모델 구조 경량화**
- 리소스 제약 환경에서 특정 환경이나 요구사항에 맞게 모델 구조 최적화 → 연산 최적화
- 새로운 작업의 입력 크기나 특징 추출 방식에 맞춰 조정
- 모델 경량화나 불필요한 연산 제거에 활용

14. 사전 학습 모델 활용_2

14.1 총별 동결 전략 비교

- Full Freeze

- 방법: Backbone 전체를 동결하고 **Head(마지막 FC 레이어)**만 학습
- 적용

- 데이터가 매우 적을 때 (< 1,000장)
- Pre-training 데이터와 유사한 도메인
- **장점:** 과적합 방지, 빠른 학습
- **단점:** 제한적인 적응력

- Partial Fine-tuning

- **방법:** 후반부 일부 층만 학습, 초반부는 동결
- **적용**
 - 중간 크기 데이터 (1K~10K)
 - 도메인 유사도에 따라 학습 범위 조절(도메인 유사도가 높으면 효과↑)
 - 도메인 유사하면 마지막 1~2블록만 Fine-tune
- **장점:** 균형잡힌 성능과 효율성
- **단점:** 최적 동결 지점 찾기 어려움

- Full Fine-tuning

- **방법:** 모든 레이어 학습
- **적용**
 - 큰 데이터셋 (> 10K)
 - Pre-training과 다른 도메인
- **장점:** 데이터가 큰 경우이므로 최대 성능 달성 가능
- **단점:** 과적합 위험, 긴 학습 시간

14.2 차등 학습률 (Discriminative Learning Rates)

- 목적

- 백본(Pre-trained) 층은 **학습률 낮게**
- 새로 만든 Head(FC층)는 **학습률 높게**
 - 사전학습된 특징은 보존
 - 새로운 데이터에 빠르게 적응 가능

PyTorch 구현 포인트

- **Parameter Group** 기능 사용하여 layer별 learning rate 설정

```
optimizer = optim.SGD([
    {'params': model.layer1.parameters(), 'lr': 0.001},
    {'params': model.layer2.parameters(), 'lr': 0.001},
    {'params': model.layer3.parameters(), 'lr': 0.005},
    {'params': model.layer4.parameters(), 'lr': 0.01},
    {'params': model.fc.parameters(), 'lr': 0.01}
])
```

14.3 Gradual Unfreezing

Epoch 범위	학습되는 Layer
1~5	Head만 학습
6~10	Head + Layer4
11~15	Head + Layer4 + Layer3
16~	모든 Layer 점진적 해제

- ULMFiT에서 검증된 기법
- 초기에는 안정적으로 Head 학습
- 점진적으로 깊은 층까지 확장
- 급격한 변화 방지 → 안정적인 수렴

14.4 Small Data 최적 학습 프로토콜

Small Data의 정의

- 클래스당 < 100 samples
- 전체 < 1,000 samples
- Pre-training 데이터와 도메인 차이가 큼

Small Data 문제점

- 과적합 위험
- 일반화 성능 저하
- 클래스 불균형 가능성
- 검증 전략의 어려움

2단계 전략

1. 전체 동결 + Head 학습 (Stage 1)
2. 점진적 Layer 해제 + 낮은 LR로 미세조정 (Stage 2)

작은 데이터셋을 위한 강력한 정규화 기법

```
# Weight Decay
optimizer = optim.SGD(model.parameters(),
                        lr=0.001,
                        weight_decay=1e-4) # L2 regularization

# Dropout
nn.Dropout(p=0.5) # FC 레이어에 추가

# Early Stopping
patience = 10
if val_loss > best_val_loss: # Validation loss가 n epoch 동안 개선되지 않으면 중단
    patience_counter += 1

# Mixup / CutMix
# 이미지와 레이블을 섞어서 학습
lambda_param = np.random.beta(alpha, alpha)
mixed_image = lambda_param * image1 + (1 - lambda_param) * image2

# Label Smoothing
criterion = nn.CrossEntropyLoss(label_smoothing=0.1) # Hard label(0 or 1) 아닌 Soft label 사용
```

14.5 핵심 내용 정리

- 데이터 크기에 따라 전략을 결정
- 차등 학습률(Differential LR)이 성능 향상에 매우 효과적
- Gradual Unfreezing은 소규모 데이터에 강력한 고급 기법
- Small Data → 먼저 동결 후 점진적 해제 전략 권장
- 전이학습(Pre-trained Model)은 도메인 유사성이 높을수록 효과 극대화

- **상황별 전략 선택 가이드**

- 데이터 적고 도메인 다름 → Full Freeze 중심 + 증강 강화
- 데이터 중간, 도메인 유사 → Partial Fine-Tuning + Differential LR
- 데이터 크고 도메인 유사 → Full Fine-Tuning

- **도메인이 다를수록**

- 더 많은 층 학습
- 더 낮은 학습률
- 점진적 해제 고려

- **실험과 검증**

- Overfitting gap 모니터링
- Validation 성능 중심으로 판단
- Early Stopping으로 과적합 방지

15. 사용자 정의 데이터 활용

15.1 ImageFolder

- 특정 트리 구조로 배치된 이미지 파일을 머신러닝의 입력 데이터로 자동 변환하는 PyTorch 기능
- **특징**
 - 폴더 이름이 **자동으로 라벨(label)** 로 매핑
 - 폴더 구조만 맞추면 **바로 Dataset으로 사용 가능**
 - 데이터셋 지정 부분만 변경하면 나머지 학습 코드는 그대로 사용 가능
- **구조**

```
root/
├── train/          # 훈련 데이터 폴더
│   ├── ants/      # 클래스 1 (폴더명 = 라벨)
│   │   ├── 0013035.jpg
│   │   └── 1030023514_aad5c608f9.jpg
```

```

|   |   |   ...
|   |   |   bees/      # 클래스 2
|   |   |   |   1092977343_cb42b38d62.jpg
|   |   |   |   ...
|   |   |   |   val/      # 검증 데이터 폴더
|   |   |   |   |   ants/
|   |   |   |   |   |   10308379_1b6c72e180.jpg
|   |   |   |   |   |   ...
|   |   |   |   |   |   bees/
|   |   |   |   |   |   |   1032546534_06907fe3b3.jpg
|   |   |   |   |   |   |   ...

```

구조의 규칙

train/ 폴더와 val/ (검증) 폴더 생성

각 폴더 아래에 클래스 이름으로 하위 폴더 생성

클래스 폴더 안에 해당 클래스의 이미지 파일 저장

15.2 데이터 전처리(Transform) 설정

• Train Transform (증강, 일반화 향상)

- RandomResizedCrop(224)
- RandomHorizontalFlip
- RandomErasing
- ToTensor()
- Normalize(mean, std)

→ 데이터 증강 기법을 사용하여 모델이 매 epoch마다 매번 조금씩 다른 이미지를 학습하게 함

→ 과적합 방지 및 일반화 성능 향상

• Validation Transform (고정, 일관된 평가용)

- Resize(256)
- CenterCrop(224)
- ToTensor()

- `Normalize(mean, std)`

→ 사전학습 된 모델들이 대부분 **224×224** 이미지 기준

→ "항상 같은 부분"으로 평가해야 검증 결과가 일관됨(이미지의 **중앙부**를 사용)

15.3 Dataset & DataLoader 생성

- Train: **244장**
- Val: **153장**
- Classes: **2개 (ants / bees)**
- batch size = **10**

```
# 작은 데이터에서는 batch size도 작게 설정함
trainloader = DataLoader(trainset, batch_size=10, shuffle=True)
valloader = DataLoader(valset, batch_size=10)
```

shuffle=True: 데이터 순서를 외우는 것 방지 (과적합↓)

15.4 Fine Tunning 방식

- **정의:** 사전 학습된 모델의 **모든 파라미터**를 우리의 데이터에 맞게 미세 조정하는 방식
- **구현 단계 (VGG-19-BN 사용)**

```
# Step 1: 사전 학습 모델 불러오기
model = models.vgg19_bn(pretrained=True)

# Step 2: 최종 계층 수정, 분류 클래스가 2개이므로 최종 노드의 출력을 2로 변경
num_features = model.classifier[6].in_features
model.classifier[6] = nn.Linear(num_features, 2) # 2개 클래스

# Step 3: 최적화 함수 설정, 모델의 모든 파라미터를 업데이트 대상으로 지정
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
```

- **결과**
 - **검증 정확도:** 96.08%, **훈련 정확도:** 94.26%
 - 과적합 없이 안정적인 성능

15.5 Transfer Learning 방식

- **정의:** 사전 학습 모델의 대부분의 가중치는 그대로 두고(동결), 마지막 분류 계층만 새로 학습하는 방식
- **구현 단계**

```
# Step 1: 가중치 동결, 모든 파라미터 동결
for param in model.parameters():
    param.requires_grad = False

# Step 2: 최종 계층 교체
num_features = model.classifier[6].in_features
model.classifier[6] = nn.Linear(num_features, 2)

# Step 3: 선택적 학습, 최종 계층의 파라미터만 학습
optimizer = optim.SGD(model.classifier[6].parameters(),
                        lr=0.001, momentum=0.9)
```

- **결과**
 - **검증 정확도:** 약 96%
 - 파인 튜닝과 동일한 성능 기록

항목	파인튜닝	전이학습
업데이트 범위	전체 레이어	마지막 레이어만
데이터 필요량	많을수록 유리	적어도 가능
학습 속도	느림	빠름
과적합 위험	비교적 있음	낮음
권장 상황	데이터 충분	데이터 적음